IMPLEMENTATION_PLAN

HelixKnowledge Skill Graph System — Comprehensive Implementation Plan

> **Status:** living master plan. Supersedes the short "Draft staged plan (S0–S6)" in `REQUIREMENTS.md` by expanding it into phases → tasks → subtasks with explicit evidence gates. `REQUIREMENTS.md` remains the source-of-truth for *what* is required (R1–R16 + the founding request + the TXT blueprint); this document is the source-of-truth for *how* and *in what order* it is built.
>
> **Author:** main orchestrator. **Date:** 2026-07-15. **Repo branch:** `main` (operator authorized direct-on-main). **Module:** `github.com/helixdevelopment/skill-system`.

---

0. How to read this plan

- Work is decomposed into **Phases (P0–P13)** and four always-on **Cross-cutting tracks (X1–X4)**. Each phase has a **Goal**, ordered **Tasks (`Pn.Tm`)**, each task has **Subtasks** and a bolded **Evidence gate** — the physical, re-runnable proof that closes the task. A task is not "done" until its evidence gate has been produced and independently re-verified by the orchestrator (anti-bluff).
- **Evidence discipline (R11 — non-negotiable):** every "works/passes/green" claim is backed by pasted, reproducible command output. No `likely/should/probably`. A freshly discovered defect halts the current task until fixed. No `--no-verify`, no force-push, no fabricated results anywhere.

- **Traceability:** the matrix in §1 maps every requirement to the phase(s) that satisfy it. Nothing may be silently dropped; if a requirement proves infeasible, it is escalated, not skipped.
- **Sizing:** Tasks are sized for one focused implementer subagent (1–3 files, clear spec) where possible; larger tasks name their split points. The subagent-driven-development loop dispatches one implementer + one task-reviewer per task, with a broad whole-branch review at each phase boundary.

---

1. Requirements traceability matrix

Req	Summary	Primary phase(s)
R1	Harden extracted Go backend to Constitution standard (build+run, tests, security, dedupe)	P0
R2	Universal on-demand dynamic skill creation; techs as working POCs; learn from real codebases	P4, P5, P14
R3	Clients: CLI, TUI, REST, Web, Desktop (all OS), Mobile (Android/iOS/HarmonyOS/Aurora); maximize shared codebase	P8
R4	Interop: Claude Code (toolkit+aliases), OpenCode, Kimi Code, HelixTrack, HelixAgent, HelixLLM	P3, P6, P9
R5	In-depth incorporation	P0 research (done) + per-phase spikes

Req	Summary	Primary phase(s)
	research for every integration	
R6	Wizard: user enters a tech set → create → map (DAG) → full processing	P5, P8
R7	Model access via LLMsVerifier / HelixLLM / Claude toolkit aliases; pluggable ModelProvider; vendor deps under `submodules/<snake_case>/`	P3, X1
R8	Exhaustive testing: 13 Constitution test types + Challenges + HelixQA; coverage → ~100%	P11 + every phase
R9	Submodule resolution: parent-dir priority else clone under `submodules/`; keep in sync	X1
R10	Fully incorporate Docs Chain submodule	P13, X1
R11	ZERO false/faulty/ bluff anywhere; positive-evidence-only	Cross-cutting (all)
R12	OpenDesign for all client design;	P10, X4

Req	Summary	Primary phase(s)
	wireframes/ sketches/Figma/ PDF/PSD/SVG artifacts	
R13	Validation skill corpus (~40 techs) built *by the system*	P14
R14	Every skill Git-versionable; MCP/ACP/plugins trigger real-time skill creation + graph expansion	P6, X2
R15	systemctl (user scope) integration + scripts: start/stop/ restart/status/ install/…	P12
R16	Skill granularity: atomic sub-skills wrapped by umbrella/ composite skills; typed-relationship building blocks	P1
Founding + blueprint	Go core; Gin + QUIC/HTTP3 + Brotli; **TOON** wire (JSON fallback); recursive DAG; central registrar; zero-bluff auto-growth jury; nano-detail docs + Mermaid; zip+tar.gz packaging	P1, P2, P4, P7, P13

2. Global constraints & Definition of Done

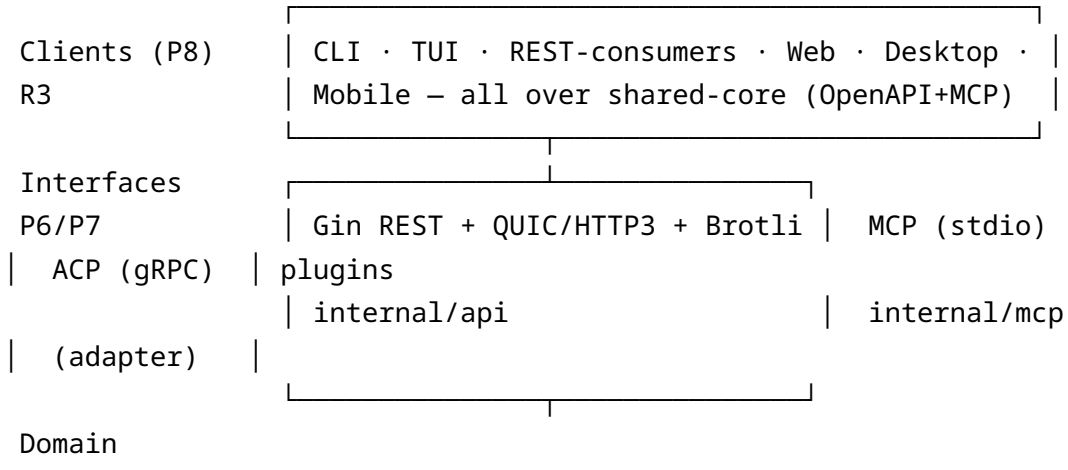
Global constraints (bind every task): 1. **Constitution inheritance** is live and gated (`tests/constitution_inheritance_gate.sh` + paired mutation). Any new submodule inherits `CLAUDE.md`/`AGENTS.md` pointers. 2. **Single-canonical dependencies** (§11.4.28C): one location per dependency; parent-ecosystem copy has priority, else `submodules/<snake_case>/`. Managed by `helix-deps.yaml` + `scripts/sync_submodules.sh` (hardened). 3. **snake_case** (§11.4.29) for dependency + skill node names. 4. **Serialization:** wire = **TOON** (`github.com/toon-format/toon`) primary + **JSON** fallback. On-disk skill files = TOML + Markdown. Config = TOML. (TOON ≠ TOML — blueprint's "interpret as TOML" is superseded.) 5. **Embedding dimension is configurable** (`config.embedding.dimensions`); the SQL `vector(N)` column and the active model MUST agree. Default pin decided in P2.T1. 6. **Zero-bluff (R11):** positive evidence only; paired mutation for every gate; no bypass flags. 7. **Git-versionable knowledge (R14):** every created/updated skill is a real commit; the repo grows as knowledge grows.

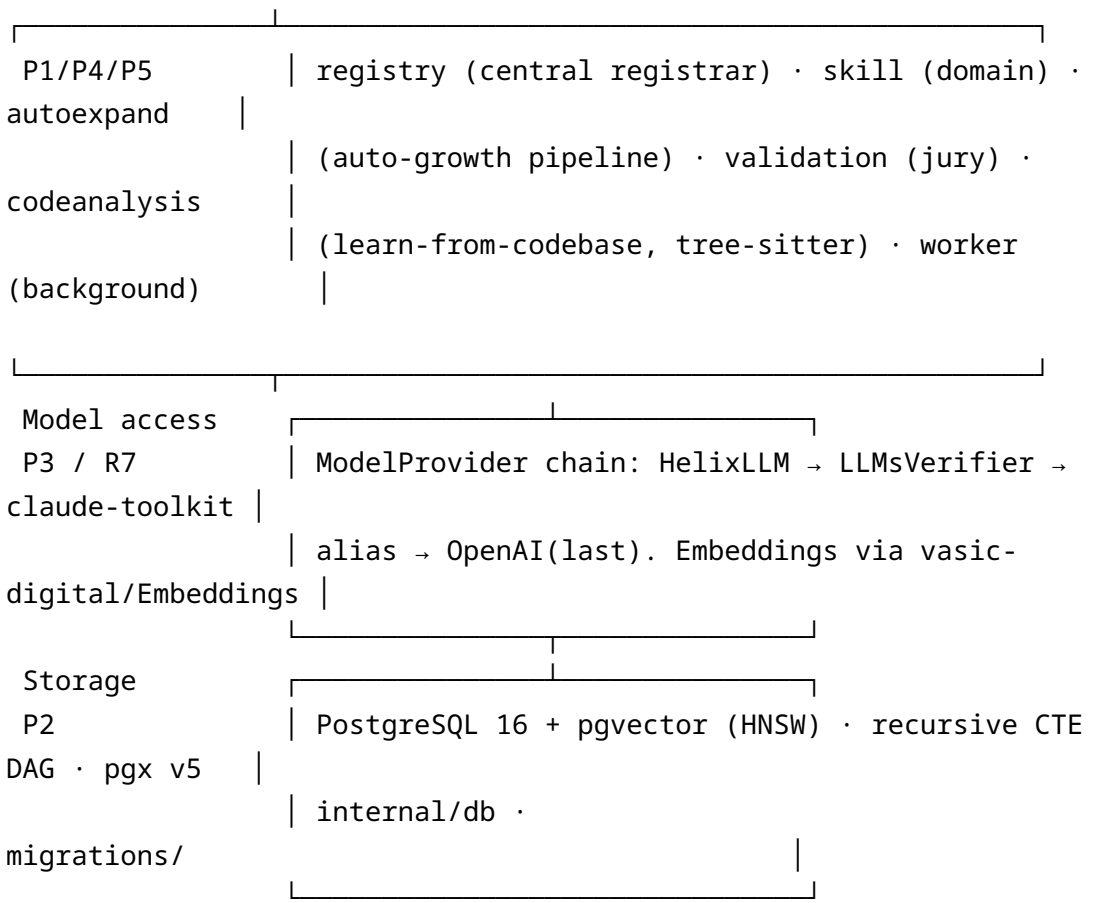
Phase Definition of Done: all tasks' evidence gates produced + re-verified; `go build ./...` and `go vet ./...` green; phase's test types (per §11) added with paired mutations and passing; broad whole-branch review clean; artifacts committed; checkpoint pushed to all upstreams **only when green**.

---

3. Architecture overview (target)

Layered (maps to real `project/internal/` packages):





Tech pins: Go 1.22+ · Gin · quic-go (HTTP/3) · andybalholm/brotli · toon-format/toon (+encoding/json) · pgx v5 + pgvector-go · tree-sitter · mcp-go (stdio) · Cobra (CLI) · Bubble Tea (TUI) · Flutter (GUI incl. HarmonyOS/Aurora) · React (Web).

P0 — Foundation hardening (R1) · *in progress*

Goal: the extracted backend compiles, vets clean, runs, has a green smoke path, one canonical DB layer, no security defects, deduped to one tree.

- **P0.T1 — Make `go build ./...` succeed.** *(Stream 1, in flight)*
  - Sub: resolve duplicate color consts (`common.go` vs `skill.go`); remove unused imports (`strconv`, `context`); fix mcp-go v0.56 API (`request.Params.Arguments → GetArguments()`); replace undefined `pgvector.RegisterTypes`/`stdlib.ConnPool`/`stdlib.GetPool` with pgx v5 + pgvector-go registration; remove `tls.Config.Allow0RTT`; drop unused `selectedStyle`.

    - Sub: pick **ONE** DB layer = **pgx v5** + **pgvector-go**; purge the 4 stray ORMs (`ent`, `gorm`, `uptrace/bun`, `go-pg`, `sqlx`) from `go.mod`; `go mod tidy`; commit a real `go.sum`.
    - **Evidence gate:** `go build ./... ; echo $?` = 0 and `go vet ./... ; echo $?` = 0, pasted; `go.mod` shows a single DB driver.
- **P0.T2 — Security defects.**
    - Sub: `internal/api/middleware.go` CORS reflect-origin + `Allow-Credentials:true` → strict allow-list from config; stop logging `api_key` query param.
    - Sub: `scripts/sync_submodules.sh` fail-closed validation — **DONE (c473d01)**, paired attack proof captured.
    - **Evidence gate:** re-run automated review → 0 HIGH/MED; unit test asserting disallowed origin is rejected + api_key never logged (paired: mutation that reintroduces reflect-origin fails the test).
- **P0.T3 — Unit-test bootstrap.** minimal `go test ./...` harness so every later package lands with tests. **Evidence gate:** `go test ./...` runs (even if few tests) exit 0.
- **P0.T4 — Smoke run.** `docker compose up` (Postgres+pgvector) + server boots + `/healthz` 200. **Evidence gate:** pasted `curl -s localhost:PORT/healthz` = 200 and `SELECT 1` via pgx.
- **P0.T5 — Dedupe to one canonical tree** — **DONE (18b3b29)**; one `project/` tree.

P0.5 — CRITICAL remediation from the gaps register (R17) · *new, top priority*

Goal: close the confirmed CRITICAL/HIGH findings in `GAPS_AND_RISKS_REGISTER.md` before building further — the green build proved SOURCE compiles, NOT that the runtime uses the hardened code (§11.4.108). These are Go-source changes with cross-package coupling, so they run **serialized** (one Go-mutator at a time), each proven by build+vet+test + the relevant D-tests + a runtime-wiring assertion.

- **P0.5.G01 — Wire `internal/api` as the single HTTP surface.** `cmd/server/main.go` runs an ad-hoc router (wildcard CORS `main.go:364`, no auth) and never imports `internal/api` (0 non-test importers). Fix: construct+run `internal/api.Server`; delete the ad-hoc router + wildcard CORS; auth fail-closed (`server.go:163`). Preserve every real endpoint (port it or report the delta — never silent-drop §11.4.122).

Evidence gate: build/vet/test green; D's CORS tests are the live server's; `grep 'Allow-Origin", "*"' cmd/server` = 0; server constructs `api.Server`.

- **P0.5.G02 — Sandbox RCE default.** the "WASM sandbox" process-fallback executes arbitrary skill code on the host with a false-isolation claim (`sandbox.go:206-290`). Fix: real isolation (rootless Podman/gVisor/true WASM) or **fail-closed SKIP**. **Evidence gate:** no host-exec path reachable by default; a hostile snippet is contained or SKIP-with-reason, proven by test.
- **P0.5.G03 — Wire the jury + auto-growth pipelines.** `internal/validation` + `internal/autoexpand` are dead code; worker handlers are stubs. Fix: wire `validation.Validate`/`autoexpand.Run` into the worker + create path. **Evidence gate:** no skill reaches `validated` without a jury verdict (gated test).
- **P0.5.G05 — Jury fail-closed on empty config.** empty jury auto-approves (`pipeline.go:428-439`). Fix: empty jury = hard error / forces human review; ≥2 real votes. **Evidence gate:** empty-jury test proves rejection, not silent pass.
- **P0.5.G06/G07 — Skill correctness.** `GetDependencyTree` truncates to depth-1 (`graph.go:306`); TOML/JSON dep+resource round-trip drops edges. Fix + prove export→import identity + full-depth tree tests.
- **P0.5.G11 — Panic-safe worker.** unchecked type assertions can crash the process; add comma-ok + per-goroutine recover.
- **P0.5.G13 — One canonical compose.** make `deploy/` canonical, delete/merge the root `docker-compose.yml`; point scripts + systemd unit at the single file.
- **P0.5.G14 — Submodule policy escalation.** §11.4.28C single-canonical vs operator parent-priority+both-synced: decision recorded (parent = canonical, `submodules/<name>` = read-only mirror); **surface to operator before any `--apply`**.
- **P0.5.G10 — Embedding dim.** pin 768 default, template `vector(N)` from config, startup dim-match assertion, OpenAI length check. (feeds P2.T1.)

P1 — Core domain model, granularity & serialization (R16 + founding)

Goal: the Skill data model supports atomic ↔ composite/umbrella granularity with typed relationship edges, serialized as TOON/JSON on the wire and TOML/Markdown on disk.

- **P1.T1 — Granularity & composition data model (R16).** *(depends on Stream A research → `research/skill_granularity_and_composition.md`)*
  - Sub: add `kind` to Skill node = `atomic | composite`; add typed edge enum (`composes/part_of`, `depends_on`, `requires`, `extends`, `prerequisite`, `related_to`, `alternative_to`) with acyclicity rules; define which edges the resolver walks.
  - Sub: extend `internal/models` structs + TOML tags; keep backward-compat with the 8 committed seed TOMLs.
  - Sub: define umbrella→component reference fields in TOML.
  - **Evidence gate:** the 8 seed TOMLs still parse; a new composite `android` umbrella + ≥4 atomic children parse and validate; unit test proves an illegal cycle on a DAG-constrained edge type is rejected (paired mutation).
- **P1.T2 — TOON serialization layer.**
  - Sub: vendor/resolve `toon-format/toon` (Go codec; implement a conformant Go marshaler/unmarshaler if no Go impl exists — with its own test vectors from the spec).
  - Sub: `internal/serialization` with `Marshal/Unmarshal` + content negotiation (`application/toon ↔ application/json`).
  - **Evidence gate:** round-trip test: struct → TOON → struct == original; struct → JSON → struct == original; a golden TOON fixture matches byte-for-byte; token-count comparison TOON vs JSON pasted.
- **P1.T3 — Skill lifecycle & versioning fields.** status (`draft→verified→merged`), semver, provenance, source refs, checksums. **Evidence gate:** unit tests for state transitions + invalid-transition rejection (paired).

P2 — Storage & DAG engine (Postgres + pgvector)

Goal: durable graph store with vector search and recursive DAG traversal.

- **P2.T1 — Schema + migrations.** finalize `vector(N)` (reconcile 768/1536/384 → pin default + make configurable); tables for skills, edges (typed), versions, embeddings; HNSW index. **Evidence gate:** `migrate up` on a fresh pgvector DB succeeds; `\d+` output pasted; HNSW index present.
- **P2.T2 — Repository layer (`internal/db`, pgx v5).** CRUD for skills/edges/versions; typed-edge inserts; pgvector-go registration. **Evidence gate:** integration tests against a real container (insert → read back identical); paired mutation (wrong dimension insert rejected).
- **P2.T3 — Recursive-CTE resolver.** "full closure needed for X" walking the resolver edge set; cycle-safe; depth-bounded. **Evidence gate:** integration test on the seed corpus returns the known android closure; a synthetic cycle does not infinite-loop (bounded + detected).
- **P2.T4 — Vector search.** semantic nearest-skill via HNSW. **Evidence gate:** query returns ranked results with distances; recall sanity check pasted.

P3 — Model access layer (R7, R4)

Goal: pluggable `ModelProvider` (never hardcode OpenAI); chain of real providers.

- **P3.T1 — `ModelProvider` interface + registry.** `Complete`, `Embed`, capabilities. **Evidence gate:** interface + fake provider unit-tested.
- **P3.T2 — Providers:** HelixLLM (OpenAI-compat `:18434`) → LLMsVerifier (`:8081`) → claude-toolkit alias shellout (`<alias> -p`) → OpenAI (last resort). Config-driven ordering + health/fallback. **Evidence gate:** each provider has a contract test (mocked HTTP / stubbed alias); fallback order proven by a test that fails provider 1 and asserts provider 2 is used.
- **P3.T3 — Embeddings via `vasic-digital/Embeddings`.** dimension must match P2.T1. **Evidence gate:** embed a fixed string → stable vector of the configured dimension.

P4 — Auto-growth validation pipeline (R2, R11, founding jury)

Goal: the zero-bluff pipeline: `draft → resource-verify → sandbox → cross-ref → LLM jury (≥2 approvals) → merge`.

- **P4.T1 — Pipeline orchestration (`internal/autoexpand` + `internal/validation`).** staged state machine, each stage gated, provenance recorded. **Evidence gate:** unit tests drive a draft through all stages; a draft failing any stage never reaches `merged` (paired).
- **P4.T2 — Resource verification.** every cited URL/source checked reachable (real HTTP/ls-remote); unverifiable content rejected. **Evidence gate:** a skill with a dead URL is rejected with the failing URL reported.
- **P4.T3 — Sandbox validation.** code/POC snippets executed in an isolated sandbox where applicable. **Evidence gate:** a POC that fails to build/run blocks merge.
- **P4.T4 — LLM jury (≥2 independent approvals).** uses P3 providers; adversarial/skeptic prompts; majority-refute kills. **Evidence gate:** a fabricated skill is rejected by the jury; an accurate one passes; both logged with juror votes.
- **P4.T5 — Merge + git commit (R14).** approved skill written to disk (TOML+MD) + DB + real git commit. **Evidence gate:** end-to-end: draft in → real commit out with the new skill file present.

P5 — Dynamic on-demand creation & learn-from-codebase (R2, R6)

Goal: create any skill on demand; learn from real codebases; wizard entrypoint.

- **P5.T1 — On-demand creation service.** given a tech name, research → draft → P4 pipeline → stored skill. **Evidence gate:** request “brotli” (not yet seeded) → produces a real, verified skill via the pipeline.
- **P5.T2 — Codebase learning (`internal/codeanalysis`, tree-sitter).** parse a real repo, extract concepts/deps into candidate skills. **Evidence gate:** run against a real sample repo → candidate skills with real extracted symbols (no lorem).
- **P5.T3 — Wizard pipeline (R6).** input set (e.g. `android`, `android_aosp`, `java`, `kotlin`, `c++`, `cmake`) → create → map (DAG) →

expand atomic closure → full processing. **Evidence gate:** end-to-end run on that exact set produces a connected sub-DAG + all closures, evidence pasted.

P6 — Interfaces: MCP + ACP + plugins, real-time growth (R14, R4)

Goal: universal agent-interop surface that triggers real-time skill creation & graph expansion.

- **P6.T1 — MCP server (`internal/mcp`, stdio).** tools: `search_skills`, `get_skill`, `expand_skill`, `create_skill`, `analyze_codebase`, `list_domain`. **Evidence gate:** MCP handshake + each tool invoked over stdio with real responses (transcript pasted).
- **P6.T2 — Real-time growth hook.** an MCP `create_skill`/`expand_skill` call runs P4/P5 live and the new node is immediately queryable. **Evidence gate:** create via MCP → immediately `get_skill` returns it in the same session.
- **P6.T3 — ACP (gRPC) adapter.** additive to MCP (research: OpenCode integrates via MCP; ACP for agents that need it). **Evidence gate:** gRPC service reflection + one round-trip call.
- **P6.T4 — Plugin trigger surface.** plugin/event → deep-research/skill-creation job enqueued to `internal/worker`. **Evidence gate:** emit event → worker creates a skill → committed.

P7 — API surface: Gin + QUIC/HTTP3 + Brotli + TOON (founding, R7)

Goal: production REST/HTTP3 API matching `api/openapi.yaml`.

- **P7.T1 — Gin handlers** for all OpenAPI paths (skills CRUD, search, expand, wizard, create, analyze). **Evidence gate:** each endpoint integration-tested; responses schema-valid.
- **P7.T2 — QUIC/HTTP3 (quic-go) + H1/H2 fallback. Evidence gate:** an HTTP/3 client round-trips a request; ALPN `h3` proven.
- **P7.T3 — Brotli compression + TOON/JSON content negotiation. Evidence gate:** `Accept: application/toon` returns TOON; `application/json` returns JSON; `Accept-Encoding: br` returns Brotli (decoded to assert equality).

- **P7.T4 — Revise `api/openapi.yaml` to TOON+JSON** (currently JSON+TOML). **Evidence gate:** OpenAPI validator passes; content types = `application/toon` + `application/json`.
- **P7.T5 — Auth** + **rate limiting** + **ApiKeyAuth** per spec. **Evidence gate:** unauthorized → 401; over-limit → 429 (tested).

P8 — Shared-core & clients (R3, R6)

Goal: one shared core; thin clients across CLI/TUI/REST/Web/Desktop/Mobile.

- **P8.T1 — Shared-core contract.** generated OpenAPI client + MCP client lib; the single source clients consume. **Evidence gate:** generated client compiles; a smoke call hits the API.
- **P8.T2 — CLI (Cobra) & TUI (Bubble Tea)** over shared-core (Go). **Evidence gate:** `helix skill get android` prints real data; TUI renders the graph (screenshot/log).
- **P8.T3 — Web (React)** over OpenAPI client. **Evidence gate:** build succeeds; wizard flow calls the API (e2e).
- **P8.T4 — Desktop (all OS)** via Flutter (or Tauri wrapping web) — decision recorded. **Evidence gate:** one desktop build artifact produced per targeted OS in CI-capable form.
- **P8.T5 — Mobile:** Android, iOS, **HarmonyOS (Flutter-OHOS)**, **Aurora (omprussia embedder)** — Flutter shared UI. **Evidence gate:** Android + iOS build; Harmony/Aurora build feasibility proven or risk-flagged with the exact blocker (no bluff).

P9 — Helix ecosystem & agent interop (R4, R5)

Goal: deep integration with HelixTrack, HelixAgent, HelixLLM (in `../helix_code/`) + Claude Code toolkit/aliases + OpenCode + Kimi Code.

- **P9.T1 — HelixLLM** as a first-class ModelProvider (already in P3) + config wiring. **Evidence gate:** live/m“cked call through HelixLLM path.
- **P9.T2 — HelixTrack / HelixAgent** integration per research spike (issue/skill sync, agent task handoff). **Evidence gate:** documented contract + one integration test/mocked round-trip.
- **P9.T3 — Claude Code:** MCP server registered + toolkit aliases; OpenCode `opencode.jsonc` MCP entry; Kimi MCP entry. **Evidence**

gate: config snippets + a real MCP connection from each client (or the exact reason one can't be tested locally).

P10 — OpenDesign design artifacts (R12, X4)

Goal: all client/design assets via OpenDesign (`git@github.com:nexu-io/open-design.git`), exported to mandatory Constitution file types.

- **P10.T1 — Vendor OpenDesign** under `submodules/open_design/` (or parent copy). **Evidence gate:** submodule resolves; license/inheritance pointers present.
- **P10.T2 — Wireframes + sketches** for CLI/TUI/Web/Desktop/Mobile + the wizard flow. **Evidence gate:** files exist and open (SVG/PDF render check).
- **P10.T3 — Figma + exports (PDF, PSD, SVG)** per Constitution mandatory design file types. **Evidence gate:** each artifact type present and validated.
- **P10.T4 — Architecture/diagram illustrations** (Mermaid + exported). **Evidence gate:** Mermaid renders; exports produced.

P11 — Exhaustive testing (R8)

Goal: all 13 Constitution test types + Challenges + HelixQA banks; coverage → ~100% where the domain permits.

- **P11.T1 — Wire the 13 test types** (per §11.4.169): unit, integration, e2e, contract, property, fuzz, mutation, performance/bench, load, security, regression, smoke, acceptance — each with paired mutations. **Evidence gate:** a runner executes all present types; the matrix of type→package is filled; paired mutations flip.
- **P11.T2 — Challenges** (`vasic-digital/Challenges`) integrated. **Evidence gate:** challenge suite runs against the system; results pasted.
- **P11.T3 — HelixQA** (`HelixDevelopment/HelixQA`) test banks wired. **Evidence gate:** HelixQA suite runs; pass/fail report pasted.
- **P11.T4 — Coverage gate.** `go test -coverprofile` → report; drive toward ~100% on domain packages; document any justified exclusions. **Evidence gate:** coverage % pasted per package; CI gate set.

P12 — Deployment & ops: systemctl (user) + Compose + scripts (R15) · *in progress*

Goal: operable via `systemctl --user` with Docker/Podman Compose and a full script set.

- **P12.T1 — Compose (pgvector/pgvector:pg16) + app service.** *(Stream B, in flight)* **Evidence gate:** `pg_isready` + `SELECT 1` + `CREATE EXTENSION vector` real output (or explicit "no engine here" + syntactic proof).
- **P12.T2 — `systemctl --user` unit** (user scope, no sudo, linger hint). *(Stream B)* **Evidence gate:** unit installs to `~/.config/systemd/user/`, `daemon-reload` clean.
- **P12.T3 — scripts/: start, stop, restart, status, install, uninstall, logs.** *(Stream B)* **Evidence gate:** `bash -n` + `shellcheck` clean on all; status-down path returns non-zero; up/down cycle proven where an engine exists.
- **P12.T4 — Reconcile compose location.** one canonical compose (existing `project/docker-compose.yml` vs new `project/deploy/`) — no rival copies. **Evidence gate:** single compose referenced by scripts + unit.

P13 — Docs, Docs Chain, packaging (R10, founding)

Goal: nano-detail documentation + Mermaid diagrams; Docs Chain incorporated; zip + tar.gz packaging.

- **P13.T1 — Docs Chain submodule** fully incorporated (`submodules/docs_chain/` or parent). **Evidence gate:** submodule resolves; docs build passes through the chain.
- **P13.T2 — Nano-detail docs**: architecture, data model, API, MCP/ACP, ops runbook, contributor guide — each with Mermaid. **Evidence gate:** docs build; every Mermaid diagram renders.
- **P13.T3 — Packaging.** reproducible `zip` + `tar.gz` of the deliverable + release script. **Evidence gate:** both archives produced; checksums; extract-and-build smoke.

P14 — Validation corpus built *by the system* (R13, R2)

Goal: create the ~40-tech corpus **through the running system** to prove it end-to-end.

- **P14.T1 — Seed baseline** — **DONE (0e0bc3b):** 8 real skills, 43-node DAG, acyclic, verified.
- **P14.T2 — Drive the full list through the wizard/pipeline:** android, android_aosp, rockchip, orange_pi, java, c, c++, kotlin, python, postgres, bash, go, gin_gonic, flutter, angular, typescript, javascript, major web/mobile/desktop frameworks, linux, macos, debugging, cmake, make, gcc, bazel, brotli, quic, http3, http, protocols, design_patterns, algorithms, security, snyk, sonarqube, maven, gradle.
  - Sub: each created via P5 on-demand + P4 jury; granulated per P1 (atomic children under umbrellas).
  - **Evidence gate:** each tech results in a real, verified, committed skill; DAG stays acyclic; per-skill provenance + real sources; a final report lists node/edge counts and any tech that could not be verified (explicitly, no bluff).

X1 — Cross-cutting: submodule resolution & sync (R7, R9, R10)

- Canonical resolver = `helix-deps.yaml` + hardened `scripts/sync_submodules.sh` (parent priority → else `submodules/<snake_case>/`).
- Keep parent + local copies in sync with main/master. **Flagged discrepancy:** Constitution §11.4.28C mandates *single-canonical*; operator asked for *parent-priority* + *both synced*. **Escalation open** — resolve before any `--apply` run that would create a second copy.
- **Evidence gate (ongoing):** every dependency resolves to exactly one canonical location; `ls-remote` reachability for each declared dep.

X2 — Cross-cutting: git-versionable knowledge growth (R14)

- Every skill create/update/merge = a real commit; repo history is the knowledge changelog.
- **Evidence gate:** after any pipeline run, `git log` shows the corresponding skill commits with real diffs.

X3 — Cross-cutting: Constitution & anti-bluff gates (R8, R11)

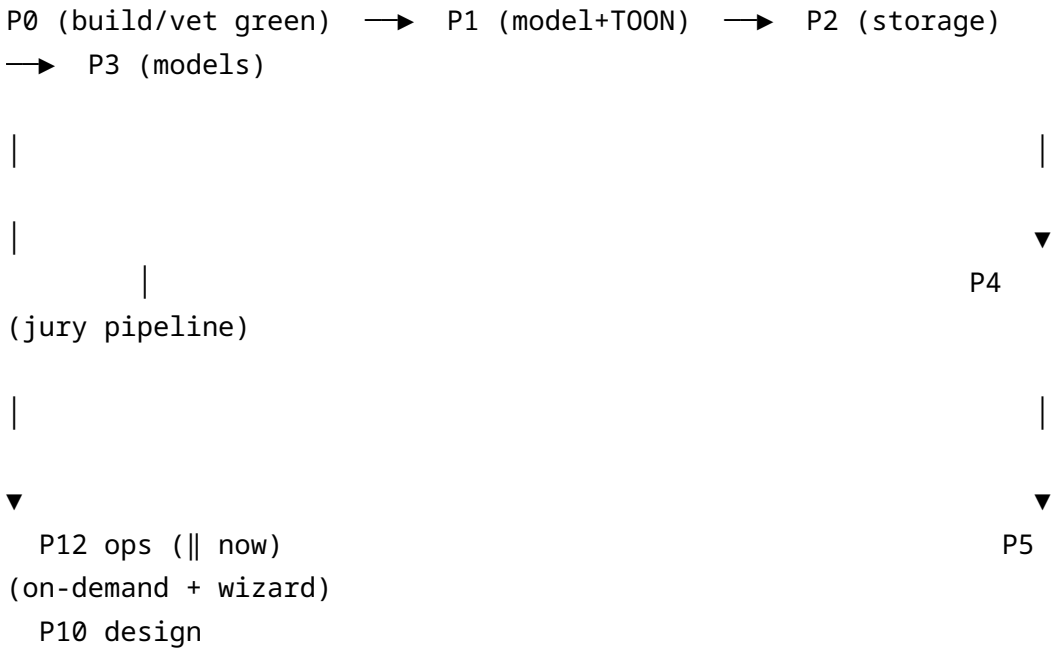
- Inheritance gate + paired mutation stays green; every new gate ships with its paired mutation; no bypass flags; freshly discovered defects halt the loop.
- **Evidence gate:** `tests/test_constitution_inheritance.sh` = PASS at every phase boundary.

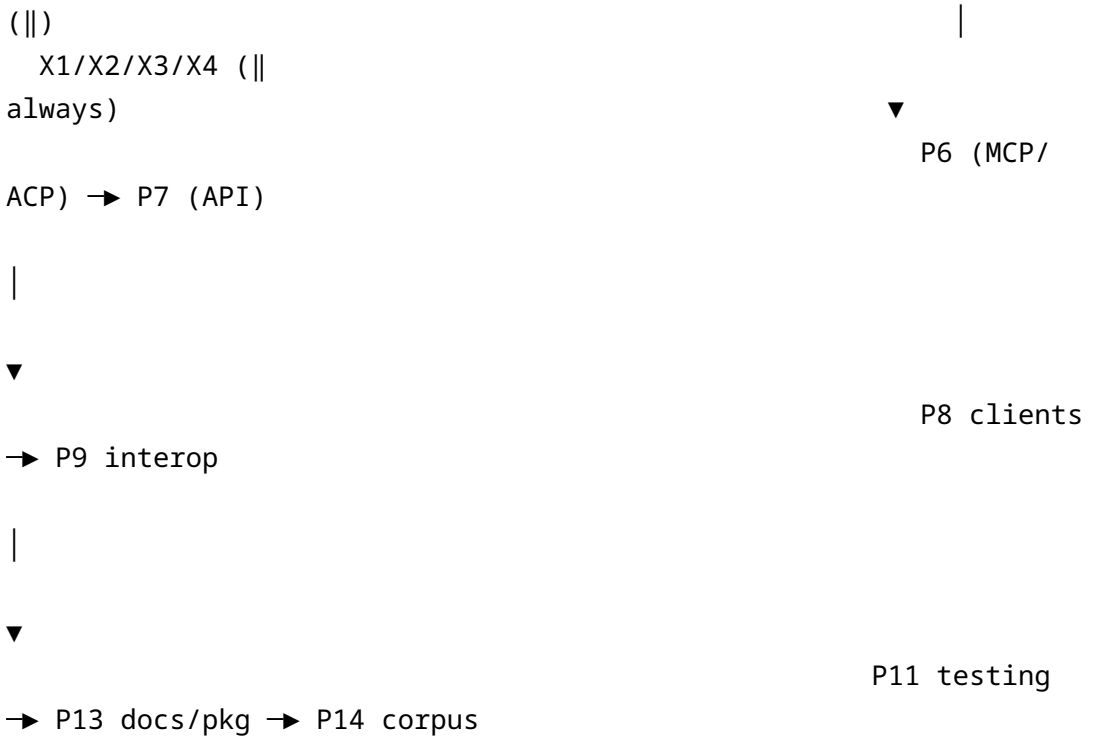
X4 — Cross-cutting: design system (R12)

- All visual/design work flows through OpenDesign; exports in mandatory file types; diagrams in Mermaid + exported.

---

4. Critical path & parallelization





- **Parallelizable now (disjoint from P0's Go build):** P12 (ops scripts), P10 (design), P1 research (granularity), X1 sync hardening — all in flight or queued.
- **Hard serial spine:** P0 → P1 → P2 → P4 → P5 → P6/P7 → P8. Clients (P8) cannot be trusted until P7 is real.
- **Testing (P11)** is not a final phase only — each phase lands its own test types; P11 is the consolidation + Challenges/HelixQA + coverage gate.

---

5. Danger zones & mitigations

1. **Aurora/HarmonyOS mobile** — smallest ecosystem, highest risk. Mitigate: Flutter-OHOS + omprussia embedder spike early in P8; risk-flag with the exact blocker rather than bluffing a build.
2. **TOON Go codec may not exist** — mitigate: implement a spec-conformant Go marshaler with the format's own test vectors (P1.T2); do not silently fall back to TOML/JSON-only.
3. **Embedding dimension drift** (768/1536/384) — mitigate: single config knob + a startup assertion that model-dim == column-dim; migration if changed.
4. **LLM jury false-approve** (R11 risk) — mitigate: ≥2 independent skeptic jurors + resource-reachability gate + sandbox; a fabricated skill must be provably rejected (P4.T4 gate).
5. **Submodule single-canonical vs parent-sync conflict** — mitigate: keep escalation X1 open; no second-copy `--apply` until resolved.

6. **Concurrent agents on one repo** — mitigate: disjoint file scopes per stream; orchestrator commits only re-verified state; upstreams get green state only.

---

6. Master deliverables checklist

- [ ] Backend builds+vets+runs; one DB layer; security clean (P0)
- [ ] Skill model with atomic/composite granularity + typed edges (P1/R16)
- [ ] TOON+JSON serialization with round-trip proof (P1/founding)
- [ ] Postgres+pgvector schema, recursive CTE, vector search (P2)
- [ ] Pluggable ModelProvider chain + embeddings (P3/R7)
- [ ] Zero-bluff auto-growth jury pipeline (P4/R2/R11)
- [ ] On-demand creation + learn-from-codebase + wizard (P5/R2/R6)
- [ ] MCP + ACP + plugins with real-time growth (P6/R14)
- [ ] Gin + QUIC/HTTP3 + Brotli + TOON API; OpenAPI (P7)
- [ ] Shared-core + CLI/TUI/Web/Desktop/Mobile incl. Harmony/Aurora (P8/R3)
- [ ] Helix ecosystem + Claude/OpenCode/Kimi interop (P9/R4)
- [ ] OpenDesign artifacts: wireframes/Figma/PDF/PSD/SVG (P10/R12)
- [ ] 13 test types + Challenges + HelixQA + coverage ~100% (P11/R8)
- [ ] systemctl –user + Compose + scripts (P12/R15)
- [ ] Docs Chain + nano-detail docs + Mermaid + zip/tar.gz (P13/R10)
- [ ] ~40-tech corpus built *by the system*, verified, committed (P14/R13)

Cross-cutting: single-canonical submodules, git-versioning, Constitution gates, design (X1–X4)

Every task above closes only on produced, re-verified, positive evidence. Upstreams receive only green, verified state. This plan is updated as phases land.